# الرسم التفاعلية AR تقرير شامل: لعبة

## دليل تقني لبناء لعبة الواقع المعزز مع الذكاء الاصطناعي

---

## 📋 جدول المحتويات


1. المستودعات المشابهة
2. معمارية اللعبة
3. دليل التقنيات
4. خطوات البناء
5. الموارد والروابط


---

## 🔍 المستودعات المشابهة

### 1. Sketch2Scene ⭐ (الأكثر صلة)

**الرابط:** https://xrvisionlabs.github.io/Sketch2Scene/

**الوصف:** مشروع بحثي من Tencent XR Vision Labs يقوم بتحويل الرسومات اليدوية البسيطة إلى عوالم ثلاثية الأبعاد تفاعلية كاملة. هذا المشروع يطابق تماماً ما تبحث عنه!

**المميزات:**

- تحويل الرسم إلى صورة Isometric (منظور متساوي القياس)
- توليد عوالم ثلاثية الأبعاد كاملة مع تضاريس وكائنات
- دعم Unity للتكامل مع محركات الألعاب
- نتائج تفاعلية وقابلة للعب

**المكونات الرئيسية:**

```
1. Sketch-Conditional Isometric Image Generation
   - استخدام ControlNet المعدل
   - Sketch-Aware Loss (SAL) للتدريب

2. Visual Scene Understanding
   - Grounded Segment Anything (GSAM) للتقسيم
   - Depth-Anything لاستخراج العمق
   - SAM + Osprey لاستخراج الخريطة

3. Procedural 3D Scene Generation
   - Unity للتجسيم والعرض
   - استخراج Mesh من الصور
   - إضافة الكائنات والشخصيات
```

**التقنيات المستخدمة:**

- ControlNet (نموذج توليد الصور)
- CLIP (التعرف على الأشياء)
- Depth-Anything (استخراج العمق)
- Unity 3D (محرك الألعاب)
- Poisson Reconstruction (بناء الشبكات)

**كيفية الاستخدام:**

1. رسم مسار بسيط (مثل خط للقطار)
2. إدخال وصف نصي اختياري
3. يقوم ControlNet بتوليد صورة Isometric
4. يتم استخراج المعلومات (Heightmap, Texture)
5. توليد عالم ثلاثي الأبعاد في Unity

---

## 2. Vibe-Draw 🎨

**الرابط:** https://github.com/martin226/vibe-draw

**الوصف:** تطبيق ويب حديث يحول الرسومات الخشنة إلى عوالم ثلاثية الأبعاد جميلة. يستخدم Next.js و Three.js مع واجهة مستخدم سهلة.

**المميزات:**

- رسم حر على Canvas
- تحسين الرسومات تلقائياً
- تحويل إلى نماذج ثلاثية الأبعاد
- بناء عوالم ثلاثية الأبعاد
- تصدير بصيغة glTF

**المعمارية:**

```
Frontend:
├── Next.js + React
├── Three.js (عرض 3D)
├── TLDraw (قماش الرسم)
└── Zustand (إدارة الحالة)

Backend:
├── FastAPI
├── Celery (مهام غير متزامنة)
├── Redis (تخزين النتائج)
└── SSE (تحديثات فورية)
```

**المتطلبات:**

- Node.js 18+
- Python 3.10+
- مفاتيح API: Claude, Gemini, Cerebras, PiAPI

**الخطوات:**

```
# إعداد Frontend
cd frontend
npm install
npm run dev

# إعداد Backend
cd backend
cp .env.example .env
docker compose up
```

**نقاط القوة:**

- واجهة مستخدم حديثة وسهلة الاستخدام
- معالجة غير متزامنة للعمليات الثقيلة
- تصدير بصيغ معيارية

---

## 3. SketchyScene 📊

**الرابط:** https://github.com/SketchyScene/SketchyScene

**الوصف:** مشروع بحثي من ECCV 2018 يوفر مجموعة بيانات ضخمة من الرسومات المشروحة بشكل غني وأدوات للتقسيم الدلالي والعينات.

**المميزات:**

- مجموعة بيانات ضخمة: 7,265 رسمة مشهد
- أدوات للتقسيم الدلالي (Semantic Segmentation)
- أدوات للتقسيم العينات (Instance Segmentation)
- أداة USketch لجمع البيانات
- DuCrawler لتنزيل الصور

**المجموعات المتاحة:**

```
1. SketchyScene-7k (750 MB)
   - 5,617 للتدريب
   - 535 للتحقق
   - 1,113 للاختبار

2. SketchyScene-Selected3 (1.6 GB)
   - 21,795 رسمة مركبة

3. SketchyScene-Synthesized30 (16.1 GB)
   - 217,950 رسمة مركبة
```

**الاستخدام:**

```
# التقسيم الدلالي
python3 segment_main.py --mode=train
python3 segment_main.py --mode=val --dcrf=0

# التقسيم العينات
python3 segment_train.py --init_model='coco'
python3 segment_evaluate.py --dataset='test'
```

**نقاط القوة:**

- مجموعة بيانات ضخمة وموثقة
- نماذج مدربة مسبقاً
- أدوات جاهزة للاستخدام

---

## 4. Robotic LEGO Assembly 🤖

**الرابط:** https://github.com/intelligent-control-lab/Robotic_LEGO_Assembly_and_Disassembly_from_Human_Demonstration

**الوصف:** مشروع متقدم يستخدم الروبوتات للتعامل مع قطع LEGO وتجميعها. يوفر نموذج للتفاعل الفيزيائي مع الكائنات.

**المميزات:**

- تجميع وتفكيك LEGO بواسطة روبوت

- رؤية حاسوبية للتعرف على القطع
- نقاط الالتقاط (Snap Points)
- محاكاة فيزيائية

**التقنيات:**

- Computer Vision (OpenCV, YOLO)
- Robot Control (FANUC)
- Physics Simulation (PyBullet)
- Motion Planning

**الفائدة لمشروعك:** يمكن استخدام نفس مبادئ "Snap Points" لتجميع LEGO في اللعبة بشكل تفاعلي.

---

## 5. Auxili.AR 📱

**الرابط:** https://github.com/lucaslrolim/Auxili.AR

**الوصف:** تطبيق AR على Android يقوم بإنشاء أدلة ثلاثية الأبعاد تفاعلية. تم تطويره كإثبات مفهوم باستخدام LEGO.

**المميزات:**

- دعم AR Foundation
- تفاعل مع الكائنات الثلاثية الأبعاد
- أدلة تفاعلية
- دعم LEGO

**المعمارية:**

```
Android App
├── AR Foundation
├── ARCore/ARKit
├── 3D Models
└── Interaction System
```

**نقاط القوة:**

- تطبيق حقيقي على الهاتف
- دعم AR كامل
- تفاعل سلس

---

### 6. BrickGPT 🧱

**الرابط:** https://avalovelace1.github.io/LegoGPT/

**الوصف:** نموذج ذكاء اصطناعي يولد هياكل LEGO مستقرة فيزيائياً من أوصاف نصية.

**المميزات:**

- توليد من نص إلى LEGO
- ضمان الاستقرار الفيزيائي
- توليد نماذج قابلة للبناء

**التقنيات:**

- GPT-based Generation
- Physics Simulation
- Constraint Solving

**الفائدة:** يمكن استخدام هذا النموذج لتوليد هياكل LEGO تفاعلية من الأوصاف.

---

# 🏗️ معمارية اللعبة

## المكونات الرئيسية

```
┌──────────────────────────────────────────────────────────┐
│                    AI-AR Sketch World                    │
├──────────────────────────────────────────────────────────┤
│                                                          │
│   ┌──────────────────────────────────────────────────┐   │
│   │          Frontend (Web/Mobile)                   │   │
│   │   ┌──────────────────────────────────────────┐   │   │
│   │   │  Canvas Drawing Interface                │   │   │
│   │   │  - رسم مسارات وأشكال                     │   │   │
│   │   │  - اختيار من عناصر جاهزة                 │   │   │
│   │   └──────────────────────────────────────────┘   │   │
│   │   ┌──────────────────────────────────────────┐   │   │
│   │   │  3D Preview (Three.js)                   │   │   │
│   │   │  - عرض العالم الثلاثي الأبعاد            │   │   │
│   │   │  - تفاعل مع الكائنات                     │   │   │
│   │   └──────────────────────────────────────────┘   │   │
│   │   ┌──────────────────────────────────────────┐   │   │
│   │   │  AR Viewer                               │   │   │
│   │   │  - عرض في الواقع المعزز                  │   │   │
│   │   │  - تتبع الحركة                           │   │   │
│   │   └──────────────────────────────────────────┘   │   │
│   └──────────────────────────────────────────────────┘   │
│                          ↓↑                              │
│   ┌──────────────────────────────────────────────────┐   │
│   │    Backend API (FastAPI/Node.js)                 │   │
│   │   ┌──────────────────────────────────────────┐   │   │
│   │   │  Sketch Processing                       │   │   │
│   │   │  - تحويل الرسم إلى Isometric             │   │   │
│   │   │  - استخراج المعلومات                     │   │   │
│   │   └──────────────────────────────────────────┘   │   │
│   │   ┌──────────────────────────────────────────┐   │   │
│   │   │  AI Models                               │   │   │
│   │   │  - ControlNet (توليد الصور)              │   │   │
│   │   │  - CLIP (التعرف على الأشياء)             │   │   │
│   │   │  - Depth-Anything (استخراج العمق)        │   │   │
│   │   └──────────────────────────────────────────┘   │   │
│   │   ┌──────────────────────────────────────────┐   │   │
│   │   │  3D Generation                           │   │   │
│   │   │  - Meshy.ai API                          │   │   │
│   │   │  - Procedural Generation                 │   │   │
```

```
│   │   └──────────────────────────────────────┘   │   │
│   └──────────────────────────────────────────────┘   │
│                          ↓↑                          │
│   ┌──────────────────────────────────────────────┐   │
│   │    Game Engine (Unity)                       │   │
│   │   ┌──────────────────────────────────────┐   │   │
│   │   │  AR Foundation                       │   │   │
│   │   │  - ARCore/ARKit                      │   │   │
│   │   │  - تتبع المستوى                      │   │   │
│   │   └──────────────────────────────────────┘   │   │
│   │   ┌──────────────────────────────────────┐   │   │
│   │   │  Physics Engine                      │   │   │
│   │   │  - تفاعل الكائنات                    │   │   │
│   │   │  - تجميع LEGO                        │   │   │
│   │   └──────────────────────────────────────┘   │   │
│   │   ┌──────────────────────────────────────┐   │   │
│   │   │  Animation System                    │   │   │
│   │   │  - حركة الشخصيات                     │   │   │
│   │   │  - حركة المركبات                     │   │   │
│   │   └──────────────────────────────────────┘   │   │
│   └──────────────────────────────────────────────┘   │
│                                                      │
└──────────────────────────────────────────────────────┘
```

## تدفق البيانات

```
1. المستخدم يرسم ← Canvas Input
                    ↓
2. إرسال الرسم ← HTTP Request
                    ↓
3. معالجة بـ ControlNet ← ControlNet API
                    ↓
4. توليد صورة Isometric ← Generated Image
                    ↓
5. استخراج المعلومات ← CLIP + Depth-Anything
                    ↓
6. توليد 3D ← Meshy.ai / Procedural Gen
                    ↓
7. عرض في Unity ← 3D Model
                    ↓
8. تفاعل المستخدم ← AR Interaction
```

---

# دليل التقنيات 🛠️

## التقنيات المستخدمة

| المكون | التقنية | الوصف |
|---|---|---|
| **Frontend** | React/Next.js | واجهة مستخدم تفاعلية |
| **الرسم** | Canvas API | رسم على الويب |
| **عرض 3D** | Three.js | عرض نماذج ثلاثية الأبعاد |
| **معالجة الرسم** | ControlNet | تحويل الرسم إلى صور |
| **التعرف** | CLIP | تصنيف الأشياء |
| **استخراج العمق** | Depth-Anything | استخراج خريطة العمق |
| **توليد 3D** | Meshy.ai | توليد نماذج 3D |
| **محرك الألعاب** | Unity | تطوير اللعبة |
| **AR** | AR Foundation | دعم الواقع المعزز |
| **الفيزياء** | PhysX | محاكاة فيزيائية |
| **Backend** | FastAPI | API سريع وفعال |
| **المهام غير المتزامنة** | Celery | معالجة العمليات الثقيلة |

### الخدمات الخارجية

```
1. Hugging Face Inference API
   - ControlNet
   - CLIP
   - Depth-Anything

2. Meshy.ai API
   - 3 نماذج توليد D من الصور

3. Sketchfab API
   - 3 نماذج عن البحث D جاهزة

4. Firebase/AWS
   - تخزين البيانات
   - المصادقة
```

---

## 📝 خطوات البناء

### المرحلة 1: إعداد البيئة

```
# 1. تثبيت Python
python3.10 --version

# 2. تثبيت Node.js
node --version
npm --version

# 3. تثبيت Unity
# تحميل من https://unity.com/download

# 4. تثبيت Docker (اختياري)
docker --version
```

## المرحلة 2: بناء Frontend

```
# 1. إنشاء مشروع React
npx create-next-app@latest sketch-world
cd sketch-world

# 2. تثبيت المكتبات
npm install three zustand axios
npm install @react-three/fiber @react-three/drei

# 3. إنشاء مكونات الرسم
# client/components/SketchCanvas.tsx
# - قماس الرسم
# - معالجة الإدخال

# 4. إنشاء معاين 3D
# client/components/ThreeDViewer.tsx
# - عرض النماذج
# - التفاعل

# 5. تشغيل الخادم
npm run dev
```

## المرحلة 3: بناء Backend

```
# 1. إنشاء مشروع FastAPI
mkdir sketch-world-backend
cd sketch-world-backend
python3 -m venv venv
source venv/bin/activate

# 2. تثبيت المكتبات
pip install fastapi uvicorn
pip install pillow numpy opencv-python
pip install requests huggingface-hub
pip install celery redis

# 3. إنشاء نقاط النهاية
# app/main.py
# - POST /sketch/process
# - POST /sketch/to-3d
# - GET /model/{id}

# 4. تكامل Hugging Face
from huggingface_hub import InferenceClient

client = InferenceClient(
    model="stabilityai/stable-diffusion-xl-base-1.0",
    token="YOUR_HF_TOKEN"
)

# 5. تشغيل الخادم
uvicorn main:app --reload
```

## المرحلة 4: بناء لعبة Unity

```
// 1. إنشاء مشروع Unity
// File > New Project > 3D

// 2. إضافة AR Foundation
// Window > TextMesh Pro > Import TMP Essentials
// Window > XR Plugin Management > Install AR Foundation

// 3. إنشاء مشهد AR
public class ARSketchWorld : MonoBehaviour {
    public ARRaycastManager raycastManager;
    public GameObject modelPrefab;

    void Update() {
        // معالجة اللمس
        if (Input.touchCount > 0) {
            Touch touch = Input.GetTouch(0);

            if (raycastManager.Raycast(
                touch.position,
                hits,
                TrackableType.PlaneWithinPolygon)) {

                // وضع النموذج
                Instantiate(modelPrefab,
                    hits[0].pose.position,
                    hits[0].pose.rotation);
            }
        }
    }
}

// 4. إضافة نظام الفيزياء
// Add Component > Rigidbody
// Add Component > Collider

// 5. إضافة الرسوم المتحركة
// Animator Component
// Animation States

// 6. البناء والنشر
// File > Build Settings > Android/iOS
```

# 📚 الموارد والروابط

## المستودعات

- **Sketch2Scene:** https://xrvisionlabs.github.io/Sketch2Scene/
- **Vibe-Draw:** https://github.com/martin226/vibe-draw
- **SketchyScene:** https://github.com/SketchyScene/SketchyScene
- **Robotic LEGO Assembly:** https://github.com/intelligent-control-lab/Robotic_LEGO_Assembly_and_Disassembly_from_Human_Demonstration
- **Auxili.AR:** https://github.com/lucaslrolim/Auxili.AR
- **BrickGPT:** https://avalovelace1.github.io/LegoGPT/

## الخدمات والأدوات

- **Hugging Face:** https://huggingface.co
- **Meshy.ai:** https://www.meshy.ai
- **Sketchfab:** https://sketchfab.com
- **Unity:** https://unity.com
- **Three.js:** https://threejs.org

## المكتبات

- **React:** https://react.dev
- **Next.js:** https://nextjs.org
- **FastAPI:** https://fastapi.tiangolo.com
- **Three.js:** https://threejs.org
- **Zustand:** https://github.com/pmndrs/zustand

## الأوراق البحثية

- **Sketch2Scene:** https://arxiv.org/abs/2408.04567
- **SketchyScene:** https://arxiv.org/abs/1801.02542

- **ControlNet:** https://arxiv.org/abs/2302.05543
- **CLIP:** https://arxiv.org/abs/2103.14030

---

## 🎯 الخطوات التالية

### للبدء الفوري:

1. **استنساخ Vibe-Draw** وتعديله لإضافة ميزات AR
2. **استخدام Sketch2Scene** كمرجع للمعمارية
3. **دمج Unity** لتطبيق الهاتف

### للتطوير المتقدم:

1. **تدريب نموذج ControlNet** مخصص على أسلوب الرسم الخاص بك
2. **بناء API** لمعالجة الرسومات **خاص**
3. **تطوير نظام تفاعل متقدم** للعبة

### للإنتاج:

1. **استضافة الخدمات** AWS/Google Cloud على
2. **نشر التطبيق** App Store و Google Play على
3. **إضافة ميزات اجتماعية** (مشاركة العوالم، التعاون)

---

## 📞 الدعم والمساعدة

### المجتمعات

- **Reddit:** r/Unity3D, r/MachineLearning
- **Discord:** Unity, Hugging Face Communities
- **GitHub Discussions:** في كل مستودع

## الموارد التعليمية

- **Unity Learn:** https://learn.unity.com
- **Hugging Face Course:** https://huggingface.co/course
- **Three.js Documentation:** https://threejs.org/docs
- **FastAPI Tutorial:** https://fastapi.tiangolo.com/tutorial

---

**آخر تحديث:** أبريل 2026 **الإصدار:** 1.0